

ESPECIFICACIÓN TÉCNICA DE INGENIERÍA

PESCO S.A. SKU AI Gap-Filler & Taxonomy Mantener

Arquitectura: Django MVT + Gemini 2.5 Flash + MCP + RAG

Metodología: Spec-Driven Development (SDD)

20.297

TOTAL SKUS SAP

4.592

PENDIENTES FAMILIA

3.627

PENDIENTES SUBFAMILIA

1.076

PENDIENTES CLASE

1. Resumen Ejecutivo y Diagnóstico Operativo

El Maestro de Artículos extraído desde SAP ERP cuenta con 20.297 registros. La problemática principal reside en la existencia de brechas en las jerarquías taxonómicas (Clase, Familia, SubFamilia y Categoría). Esta especificación define la construcción de un sistema de resolución incremental enfocado únicamente en la clasificación de los atributos faltantes (Gap-Filling) sin reprocesar datos validados previamente.

Estrategia Anti-Alucinación: Uso combinado de un motor determinista (Regex/Pattern Match), Few-Shot Prompting con salida estructurada JSON Schema ('Gemini 2.5 Flash') y RAG local sobre los 15.700 SKUs con taxonomía completa.

2. Modelo de Datos Django ORM (`taxonomy/models.py`)

El modelo almacena el catálogo completo y añade marcas de auditoría para rastrear qué campos específicos están pendientes de clasificación.

```

from django.db import models

class SKUIItem(models.Model):
    item_code = models.CharField(max_length=50, unique=True, db_index=True)
    item_name = models.TextField()
    stock = models.FloatField(default=0.0)
    costo_un = models.DecimalField(max_digits=12, decimal_places=2, default=0.0)
    costo_tt = models.DecimalField(max_digits=12, decimal_places=2, default=0.0)
    moneda = models.CharField(max_length=10, null=True, blank=True)
    precio_lista = models.DecimalField(max_digits=12, decimal_places=2, default=0.0)

    # Atributos Taxonómicos
    cod_grupo = models.CharField(max_length=50, null=True, blank=True)
    nombre_grupo = models.CharField(max_length=100, db_index=True)
    clase = models.CharField(max_length=100, null=True, blank=True, db_index=True)
    familia = models.CharField(max_length=100, null=True, blank=True, db_index=True)
    subfamilia = models.CharField(max_length=100, null=True, blank=True, db_index=True)
    modelo = models.CharField(max_length=100, null=True, blank=True)
    categoria = models.CharField(max_length=100, null=True, blank=True)

    # Auditoría e IA
    is_incomplete = models.BooleanField(default=True, db_index=True)
    pending_fields = models.JSONField(default=list)
    ai_confidence_score = models.FloatField(default=0.0)
    ai_rationale = models.TextField(null=True, blank=True)
    updated_at = models.DateTimeField(auto_now=True)

    def check_incomplete(self):
        missing = []
        if not self.clase: missing.append("clase")
        if not self.familia: missing.append("familia")
        if not self.subfamilia: missing.append("subfamilia")
        if not self.categoria: missing.append("categoria")
        self.pending_fields = missing
        self.is_incomplete = len(missing) > 0
        return self.is_incomplete

```

3. Agente de Clasificación & Gemini 2.5 Flash (`ai\_engine/gap\_classifier.py`)

Invocación estructurada a la API de Google Gemini utilizando Pydantic para restringir el output y evitar generación de clases inexistentes.

```

import json
import google.generativeai as genai
from pydantic import BaseModel, Field
from typing import Optional, List, Dict, Any

class DynamicTaxonomyFill(BaseModel):
    clase: Optional[str] = Field(None, description="Clase asignada si faltaba")
    familia: Optional[str] = Field(None, description="Familia asignada dentro del catálogo PESCO")
    subfamilia: Optional[str] = Field(None, description="Subfamilia/Marca asignada")
    categoria: Optional[str] = Field(None, description="Categoría operacional")
    confidence: float = Field(..., description="Nivel de certeza de 0.0 a 1.0")
    rationale: str = Field(..., description="Explicación técnica del resultado")

class GeminiGapClassifier:
    def __init__(self, api_key: str):
        genai.configure(api_key=api_key)
        self.model = genai.GenerativeModel(
            model_name="gemini-2.5-flash",
            generation_config={"response_mime_type": "application/json", "temperature": 0.0}
        )

    def fill_missing_fields(self, item_name: str, grupo: str, current_data: Dict[str, Any],
        missing_fields: List[str], rag_context: List[Dict]) -> DynamicTaxonomyFill:
        examples_str = "\n".join([f"- Similar: {item['item_name']} -> Familia: {item['familia']},
        SubFamilia: {item['subfamilia']}" for item in rag_context])
        prompt = f'''
Eres un especialista en catalogación de maquinaria pesada y repuestos para PESCO S.A.
Rellena EXCLUSIVAMENTE los campos faltantes: {missing_fields}

SKU: {item_name} | Grupo: {grupo} | Datos Actuales: {json.dumps(current_data, ensure_ascii=False)}
Ejemplos RAG Reales:
{examples_str}

Responde siguiendo la estructura JSON:
{DynamicTaxonomyFill.schema_json()}
'''

        response = self.model.generate_content(prompt)
        return DynamicTaxonomyFill(**json.loads(response.text))

```

4. Pipeline de Procesamiento Masivo (`taxonomy/management/commands/process\_pending\_skus.py`)

```
from django.core.management.base import BaseCommand
from taxonomy.models import SKUItem
from ai_engine.gap_classifier import GeminiGapClassifier
import time

class Command(BaseCommand):
    help = "Procesa únicamente los SKUs que tienen atributos incompletos."

    def add_arguments(self, parser):
        parser.add_argument('--api-key', type=str, required=True)
        parser.add_argument('--limit', type=int, default=500)

    def handle(self, *args, **options):
        classifier = GeminiGapClassifier(api_key=options['api_key'])
        incomplete_skus = SKUItem.objects.filter(is_incomplete=True)[:options['limit']]

        for sku in incomplete_skus:
            rag_matches = list(SKUItem.objects.filter(
                is_incomplete=False, nombre_grupo=sku.nombre_grupo
            ).values('item_name', 'familia', 'subfamilia')[:3])

            current_data = {"clase": sku.clase, "familia": sku.familia, "subfamilia": sku.subfamilia,
                           "categoria": sku.categoria}

            try:
                result = classifier.fill_missing_fields(sku.item_name, sku.nombre_grupo, current_data,
                                                       sku.pending_fields, rag_matches)
                if "clase" in sku.pending_fields and result.clase: sku.clase = result.clase
                if "familia" in sku.pending_fields and result.familia: sku.familia = result.familia
                if "subfamilia" in sku.pending_fields and result.subfamilia: sku.subfamilia =
result.subfamilia
                if "categoria" in sku.pending_fields and result.categoria: sku.categoria =
result.categoria
                sku.ai_confidence_score = result.confidence
                sku.ai_rationale = result.rationale
                sku.check_incomplete()
                sku.save()
            except Exception as e:
                self.stderr.write(f"Error en SKU {sku.item_code}: {str(e)}")
```

5. Servidor MCP para Auditoría Externa (`mcp\_server/pesco\_mcp.py`)

```
from mcp.server.fastmcp import FastMCP
import sqlite3

mcp = FastMCP("PESCO Taxonomy Auditor MCP")

@mcp.tool()
def get_pending_summary() -> str:
    conn = sqlite3.connect("db.sqlite3")
    cursor = conn.cursor()
    cursor.execute('''
        SELECT
            SUM(CASE WHEN clase IS NULL OR clase = '' THEN 1 ELSE 0 END),
            SUM(CASE WHEN familia IS NULL OR familia = '' THEN 1 ELSE 0 END),
            SUM(CASE WHEN subfamilia IS NULL OR subfamilia = '' THEN 1 ELSE 0 END),
            COUNT(*)
        FROM taxonomy_skuitem
    ''')
    row = cursor.fetchone()
    conn.close()
    return f"Total: {row[3]} | Sin Clase: {row[0]} | Sin Familia: {row[1]} | Sin SubFamilia: {row[2]}"

if __name__ == "__main__":
    mcp.run()
```

6. Pruebas Unitarias Automatizadas con Pytest (`tests/test\_gap\_classifier.py`)

```
import pytest
from taxonomy.models import SKUItem

@pytest.mark.django_db
def test_incomplete_flag_behavior():
    sku = SKUItem.objects.create(
        item_code="PESCO-001",
        item_name="FILTRO DE ACEITE ROSENBAUER",
        clase="Repuestos",
        familia=None,
        subfamilia="ROSENBAUER"
    )
    sku.check_incomplete()
    assert sku.is_incomplete is True
    assert "familia" in sku.pending_fields
    assert "clase" not in sku.pending_fields
```

7. Guía de Despliegue y Ejecución Paso a Paso

Paso	Comando	Descripción
1. Migración DB	<code>python manage.py migrate</code>	Prepara las tablas SQLite con soporte para JSONField.
2. Carga Inicial	<code>python manage.py load_sap_excel --file "2907...xlsx"</code>	Lee los 20.297 registros marcando sólo los incompletos.

Paso	Comando	Descripción
3. Ejecutar Agente IA	<code>python manage.py process_pending_skus --api-key "API_KEY"</code>	Inicia el proceso incremental con Gemini 2.5 Flash.
4. Test Harness	<code>pytest</code>	Valida la consistencia antes de enviar a producción.